Metode Numerice @ CS-UPB

Echipa MN CS - 2015:

Profesori:

- Pantelimon George Popescu - Seria CA;
- Florin Pop - Seria CB, CC;

Asistenti:

PG Popescu, Clementin Cercel, Sorin Ciolofan, Bogdan Tiganoaia, Diana Udrea, Alexandru Tifrea, David Iancu, Bogdan Cristian Marchis, Radu Poenaru, Madalina Grosu, Andrei Vlad Postoaca, Radu Constantinescu, Roxana Istrate, Madalina Hristache, Adelina Vidovici;

Regulament MN 2015

Reguli de notare

- NotaMN = NotaExamen + NotaParcurs + Bonus
- NotaExam (maxim 4 puncte) este nota obţinuta la examenul final.
- NotaParcurs (maxim 6 puncte) este nota obţinuta pentru activitatea din timpul semestrului
- Bonus (maxim 1 punct) este nota obţinuta din bonusuri curs + laborator.

Regulament MN 2015

- NotaParcurs = NotaPartial + NotaLab + NotaTeme
- NotaPartial (maxim 3 puncte) este nota obţinuta la examenul parţial. Nu se repeta.
- NotaLab (maxim 1 punct).
- NotaTeme (maxim 2 puncte) este nota obţinută din cele 4 teme (0.75 pe tema) de casă

Regulament MN 2015

Reguli de promovare

- NotaParcurs >= 3
- NotaExam >= 2

Exemplu Problema

Bungee-jumping



F=ma

F/m=a

F=Fgravitationala+Frezistenta_aer

Fgravitationala=mg

Frezistenta_aer=-cv^2

$$\frac{dv}{dt} = g - \frac{c_d}{m}v^2$$

Exemplu Problema

Bungee-jumping



Consideram urmatoarele date intitiale

m=65,1 kg;
g=9,81 m/s^2;
c=0,25 kg/m;

Ne propunem sa aflam viteza la un moment dat de timp

Exemplu Problema

Bungee-jumping Solutie ANALITICA



Solutia ecuatiei $\frac{dv}{dt} = g - \frac{c_d}{m}v^2$

este $v(t) = \sqrt{\frac{gm}{c_d}}\tanh\left(\sqrt{\frac{gc_d}{m}}t\right)$

unde tanh este tangenta hiperbolica,

$\tanh x = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

Exemplu Problema

Bungee-jumping Solutie ANALITICA



Dupa introducerea valorilor ecuatia devine

$v(t) = \sqrt{\frac{9.81(68.1)}{0.25}}\tanh\left(\sqrt{\frac{9.81(0.25)}{68.1}}t\right) = 51.6938\tanh(0.18977t)$

Exemplu Problema

Bungee-jumping Solutie ANALITICA



Si deci crescand timpul cu cate 2 secunde, in primele 12 secunde vom obtine

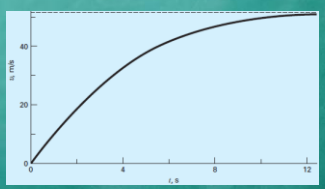
t, s	v, m/s
0	0
2	18.7292
4	33.1118
6	42.0762
8	46.9575
10	49.4214
12	50.6175
∞	51.6938

Exemplu Problema

Bungee-jumping Solutie ANALITICA



Reprezentand grafic, analitic, v(t)



Exemplu Problema

Bungee-jumping Solutie NUMERICA



Consideram $\frac{dv}{dt} \cong \frac{\Delta v}{\Delta t} = \frac{v(t_{i+1}) - v(t_i)}{t_{i+1} - t_i}$

iar ecuatia se transforma in

$$v(t_{i+1}) = v(t_i) + \left[g - \frac{c_d}{m}v(t_i)^2\right](t_{i+1} - t_i)$$

Exemplu Problema

Bungee-jumping Solutie NUMERICA



Consideram tot un pas de 2 secunde pentru a determina viteza in primele 12 secunde la un moment dat de timp.

Consideram inca de la inceput ca viteza la momentul 0 a fost 0 si deci viteza la 2 secunde va fi

$$v = 0 + \left[9.81 - \frac{0.25}{68.1}(0)^2\right] \times 2 = 19.62 \text{ m/s}$$

Exemplu Problema

Bungee-jumping Solutie NUMERICA



Acum cunoscand viteza la 2 secunde, putem afla viteza la 4 secunde,

$$v = 19.62 + \left[9.81 - \frac{0.25}{68.1}(19.62)^2\right] \times 2 = 36.4137 \text{ m/s}$$

Exemplu Problema

Bungee-jumping Solutie NUMERICA



samd, obtinand astfel urmatoarele date

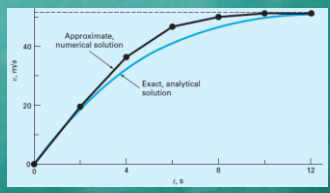
t, s	v, m/s
0	0
2	19.6200
4	36.4137
6	46.2983
8	50.1802
10	51.3123
12	51.6008
∞	51.6938

Exemplu Problema

Bungee-jumping Solutie NUMERICA



Reprezentand grafic, numeric, v(t)



The Best of the 20th Century: Top 10 Algorithms

1946: John von Neumann, Stan Ulam, and Nick Metropolis, all at the Los Alamos Scientific Laboratory, cook up the Metropolis algorithm, also known as the **Monte Carlo method**.

The Metropolis algorithm aims to obtain approximate solutions to numerical problems with unmanageably many degrees of freedom and to combinatorial problems of factorial size, by mimicking a random process. Given the digital computer's reputation for deterministic calculation, it's fitting that one of its earliest applications was the generation of random numbers.



1947: George Dantzig, at the RAND Corporation, creates the **simplex method for linear programming**.

In terms of widespread application, Dantzig's algorithm is one of the most successful of all time: Linear programming dominates the world of industry, where economic survival depends on the ability to optimize within budgetary and other constraints. (Of course, the "real" problems of industry are often nonlinear; the use of linear programming is sometimes dictated by the computational budget.) The simplex method is an elegant way of arriving at optimal answers. Although theoretically susceptible to exponential delays, the algorithm in practice is highly efficient—which in itself says something interesting about the nature of computation.

The Best of the 20th Century: Editors Name Top 10 Algorithms

1950: Magnus Hestenes, Eduard Stiefel, and Cornelius Lanczos, all from the Institute for Numerical Analysis at the National Bureau of Standards, initiate the development of **Krylov subspace iteration methods**.

These algorithms address the seemingly simple task of solving equations of the form $Ax = b$. The catch, of course, is that A is a huge $n \times n$ matrix, so that the algebraic answer $x = b/A$ is not so easy to compute. (Indeed, matrix "division" is not a particularly useful concept.) Iterative methods—such as solving equations of the form $Kx_{i+1} = Kx_i + b - Ax_i$ with a simpler matrix K that's ideally "close" to A—lead to the study of Krylov subspaces. Named for the Russian mathematician Nikolai Krylov, Krylov subspaces are spanned by powers of a matrix applied to an initial "remainder" vector $r_0 = b - Ax_0$. Lanczos found a nifty way to generate an orthogonal basis for such a subspace when the matrix is symmetric. Hestenes and Stiefel proposed an even nuftier method, known as the conjugate gradient method, for systems that are both symmetric and positive definite. Over the last 50 years, numerous researchers have improved and extended these algorithms. The current suite includes techniques for non-symmetric systems, with acronyms like GMRES and Bi-CGSTAB. (GMRES and Bi-CGSTAB premiered in *SIAM Journal on Scientific and Statistical Computing*, in 1986 and 1992, respectively.)

The Best of the 20th Century: Editors Name Top 10 Algorithms

1951: Alston Householder of Oak Ridge National Laboratory formalizes the **decompositional approach to matrix computations**.

The ability to factor matrices into triangular, diagonal, orthogonal, and other special forms has turned out to be extremely useful. The decompositional approach has enabled software developers to produce flexible and efficient matrix packages. It also facilitates the analysis of rounding errors, one of the big bugbears of numerical linear algebra. (In 1961, James Wilkinson of the National Physical Laboratory in London published a seminal paper in the *Journal of the ACM*, titled "Error Analysis of Direct Methods of Matrix Inversion," based on the LU decomposition of a matrix as a product of lower and upper triangular factors.)



Alston Householder

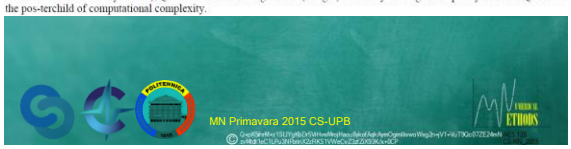
1957: John Backus leads a team at IBM in developing the **Fortran optimizing compiler**.

The creation of Fortran may rank as the single most important event in the history of computer programming: Finally, scientists (and others) could tell the computer what they wanted it to do, without having to descend into the netherworld of machine code. Although modest by modern compiler standards—Fortran I consisted of a mere 23,500 assembly-language instructions—the early compiler was nonetheless capable of surprisingly sophisticated computations. As Backus himself recalls in a recent history of Fortran I, II, and III, published in 1998 in the *IEEE Annals of the History of Computing*, the compiler "produced code of such efficiency that its output would startle the programmers who studied it."

The Best of the 20th Century: Editors Name Top 10 Algorithms

1959–61: J.G.F. Francis of Ferranti Ltd., London, finds a stable method for computing eigenvalues, known as the **QR algorithm**.
Eigenvalues are arguably the most important numbers associated with matrices—and they can be the trickiest to compute. It's relatively easy to transform a square matrix into a matrix that's "almost" upper triangular, meaning one with a single extra set of nonzero entries just below the main diagonal. But chipping away those final nonzeros, without launching an avalanche of error, is nontrivial. The QR algorithm is just the ticket. Based on the QR decomposition, which writes A as the product of an orthogonal matrix Q and an upper triangular matrix R, this approach iteratively changes $A_i = QR$ into $A_{i+1} = RQ$, with a few bells and whistles for accelerating convergence to upper triangular form. By the mid-1960s, the QR algorithm had turned once-formidable eigenvalue problems into routine calculations.

1962: Tony Hoare of Elliott Brothers, Ltd., London, presents **Quicksort**.
Putting N things in numerical or alphabetical order is mind-numbingly mundane. The intellectual challenge lies in devising ways of doing so quickly. Hoare's algorithm uses the age-old recursive strategy of divide and conquer to solve the problem: Pick one element as a "pivot," separate the rest into piles of "big" and "small" elements (as compared with the pivot), and then repeat this procedure on each pile. Although it's possible to get stuck doing all $N(N - 1)/2$ comparisons (especially if you use as your pivot the first item on a list that's already sorted!), Quicksort runs on average with $O(N \log N)$ efficiency. Its elegant simplicity has made Quicksort the pos-terchild of computational complexity.



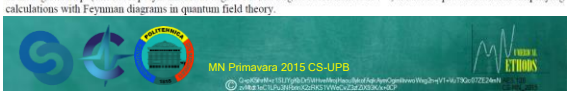
The Best of the 20th Century: Editors Name Top 10 Algorithms


James Cooley


John Tukey

1965: James Cooley of the IBM T.J. Watson Research Center and John Tukey of Princeton University and AT&T Bell Laboratories unveil the **fast Fourier transform**.
Easily the most far-reaching algo-rithm in applied mathematics, the FFT revolutionized signal processing. The underlying idea goes back to Gauss (who needed to calculate orbits of asteroids), but it was the Cooley–Tukey paper that made it clear how easily Fourier transforms can be computed. Like Quicksort, the FFT relies on a divide-and-conquer strategy to reduce an ostensibly $O(N^2)$ chore to an $O(N \log N)$ frolic. But unlike Quick- sort, the implementation is (at first sight) nonintuitive and less than straightforward. This in itself gave computer science an impetus to investigate the inherent complexity of computational problems and algorithms.

1977: Helaman Ferguson and Rodney Forcade of Brigham Young University advance an **integer relation detection algorithm**.
The problem is an old one: Given a bunch of real numbers, say $x_1, x_2, \ldots, x_n$, are there integers $a_1, a_2, \ldots, a_n$ (not all 0) for which $a_1x_1 + a_2x_2 + \ldots + a_nx_n = 0$? For $n = 2$, the venerable Euclidean algorithm does the job, computing terms in the continued-fraction expansion of x_1/x_2. If x_1/x_2 is rational, the expansion terminates and, with proper unraveling, gives the "smallest" integers a_1 and a_2. If the Euclidean algorithm doesn't terminate—or if you simply get tired of computing it—then the unraveling procedure at least provides lower bounds on the size of the smallest integer relation. Ferguson and Forcade's generalization, although much more difficult to implement (and to understand), is also more powerful. Their detection algorithm, for example, has been used to find the precise coefficients of the polynomials satisfied by the third and fourth bifurcation points, $B_3 = 3.544090$ and $B_4 = 3.564407$, of the logistic map. (The latter polynomial is of degree 120; its largest coefficient is 257^{80}.) It has also proved useful in simplifying calculations with Feynman diagrams in quantum field theory.



The Best of the 20th Century: Editors Name Top 10 Algorithms

1987: Leslie Greengard and Vladimir Rokhlin of Yale University invent the **fast multipole algorithm**.
This algorithm overcomes one of the biggest headaches of N-body simulations: the fact that accurate calculations of the motions of N particles interacting via gravitational or electrostatic forces (think stars in a galaxy, or atoms in a protein) would seem to require $O(N^2)$ computations—one for each pair of particles. The fast multipole algorithm gets by with $O(N)$ computations. It does so by using multipole expansions (net charge or mass, dipole moment, quadrupole, and so forth) to approximate the effects of a distant group of particles on a local group. A hierarchical decomposition of space is used to define ever-larger groups as distances increase. One of the distinct advantages of the fast multipole algorithm is that it comes equipped with rigorous error estimates, a feature that many methods lack.

*January/February 2000,*Computing in Science & Engineering*, a joint publication of the American Institute of Physics and the IEEE Computer Society. Guest editors Jack Dongarra of the University of Tennessee and Oak Ridge National Laboratory and Francis Sullivan of the Center for Comput-ing Sciences at the Institute for Defense Analyses put togeth-er a list they call the "Top Ten Algorithms of the Century."

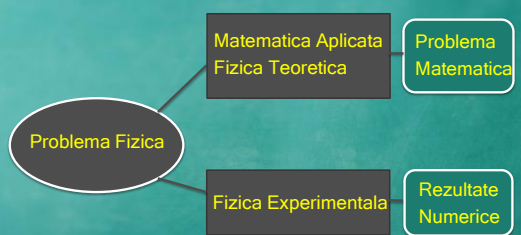


Continut curs MN

- Introducere.
- Despre matrici.
- Transformari.
- Sisteme de ecuatii lineare.
- Ecuatii neliniare.
- Valori proprii şi vectori proprii.
- Interpolare polinomiala.
- Teoria Aproximarii.
- Integrare si derivare numerica.
- Integrarea ecuatiilor diferenţiale ordinare. (ODE)

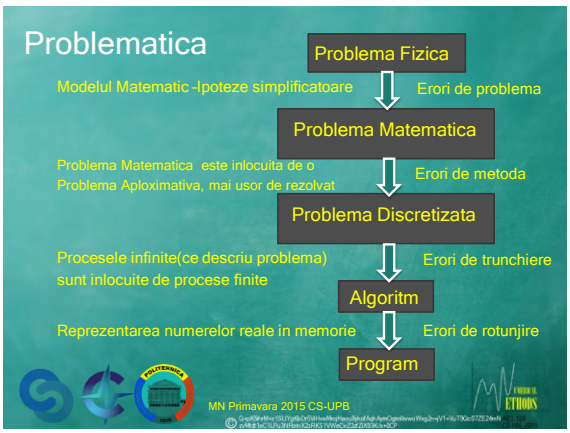


Problematica





Problematica



Virgula mobila IEEE754

Virgula mobila

$fl(x) = \pm 0.a_1a_2...a_t \times \beta^e$
reprezentare normalizată
$1 \le a_1 < \beta$ şi $0 \le a_i < \beta$, $i=2:t$
$L \le e \le U$

- *Sistemul de reprezentare în virgulă mobilă* $F(\beta, t, L, U)$ cuprinde:
- baza β
- precizia reprezentării t
- limitele (superioară şi inferioară ale) exponentului L şi U
- reprezentarea lui zero

Exemplu: $F(10, 1, 0, 1) = \{ \pm 0.a_1 \times 10^e \} \cup \{0\}$ cu $a_1 \in \{1:9\}$ şi $e \in \{0,1\}$, în total 37 de numere.

Virgula mobila IEEE754

Virgula mobila

- $2(\beta-1)\beta^{t-1}(U-L+1)$ valori distincte
- a_1 poate lua $\beta-1$ valori distincte,
- restul de $t-1$ cifre poate lua fiecare β valori diferite, deci β^{t-1},
- exponentul ia $U-L+1$, şi
- semnul două).

Cel mai mare număr reprezentabil Ω, (realmax) are forma:

$\Omega = 0.(\beta-1)(\beta-1)...(\beta-1) \times \beta^U =$
$= [(\beta-1)/\beta^1 + (\beta-1)/\beta^2 + ... + (\beta-1)/\beta^t] \times \beta^U$
$= (\beta-1)/\beta(1-\beta^{-t})/(1-\beta^{-1}) \times \beta^U = \beta^U(1-\beta^{-t})$
$\Omega = \beta^U(1-\beta^{-t})$

- Cel mai mic număr pozitiv reprezentabil ω numit şi realmin este:

$\Omega = 0.10...0 \times \beta^L = \beta^L/\beta = \beta^{L-1}$
$\omega = \beta^{L-1}$

Virgula mobila IEEE754

Surse erori

Un număr real $x \in F$ se reprezintă exact, dacă suma se termină înainte de t termeni şi exponentul este cuprins între limite. Altfel, numărul real x se ***aproximează*** printr-o valoare $fl(x) \in F$
Aproximarea numărului real
$x = (0.a_1a_2...)\beta^e = (a_1\beta^{-1} + a_2\beta^{-2} + ... + a_t\beta^{-t} + a_{t+1}\beta^{-t-1} + ...)\beta^e$
se poate face prin ***trunchiere*** sau prin ***rotunjire.***

- Aproximarea prin trunchiere ignoră cifrele numărului real din dreapta poziției t.

$fl(x) = (0.a_1a_2...a_t)\beta^e = (a_1\beta^{-1} + a_2\beta^{-2} + ... + a_t\beta^{-t})\beta^e$

- Aproximarea prin rotunjire consideră:

$fl(x) = (0.a_1a_2...a_t + 1)\beta^e$ dacă $a_{t+1} \ge \beta/2$
$fl(x) = (0.a_1a_2...a_t)\beta^e$ dacă $a_{t+1} < \beta/2$

O ***depăşire superioară*** apare dacă $e > U$.
Ea declanşează o *eroare la execuţie*, care conduce la întreruperea calculelor.
O ***depăşire inferioară*** apare dacă $e < L$;
ea duce la înlocuirea numărului prin zero.

Virgula mobila IEEE754

Surse erori

- ***Epsilon maşină*** (notat eps în Matlab sau μ) reprezintă cel mai mic număr pozitiv cu proprietatea că:

$fl(1+\mu) > 1$

De exemplu în $F(10, 4, -3, 3)$ cu ***rotunjire prin tăiere (trunchiere)***:

- $fl(1+0.0009) = fl(1.0009) = 1$
- $fl(1+0.0010) = fl(1.0010) = 1.001 > 1$

aşadar $\mu_{tr} = 0.001 = 10^{-3} > \omega = 10^{-4}$.

- Dacă se foloseşte rotunjire, atunci:

$fl(1+0.0004) = fl(1.0004) = 1$
$fl(1+0.0005) = fl(1.0005) = 1.001 > 1$
cu $\mu_{zot} = 0.0005 = 1/2.10^{-3} = 1/2.\mu_{tr}$

Virgula mobila IEEE754

Surse erori

- ***Eroarea absolută*** la rotunjirea prin trunchiere:

$e_x = x - fl(x) = (a_1/\beta^1 + a_2/\beta^2 + ... + a_t/\beta^t + a_{t+1}/\beta^{t+1} + ...)\beta^e$
$\quad - (a_1/\beta^1 + a_2/\beta^2 + ... + a_t/\beta^t)\beta^e$
$e_x = (a_{t+1}/\beta^1 + a_{t+2}/\beta^2 + ...)\beta^{e-t}$

$|e_x| \le |(\beta-1)/\beta^1 + (\beta-1)/\beta^2 + ...|\beta^{e-t} =$
$(\beta-1)\beta^{e-t}\ (1/\beta^1 + 1/\beta^2 + ...) \le (\beta-1)\beta^{e-t}/(\beta-1) = \beta^{e-t}$
$|e_x| \le \beta^{e-t}$

Dacă se foloseşte **rotunjirea** atunci eroarea absolută este şi mai mică:

$|e_x| \le 1/2.\ \beta^{e-t}$

Virgula mobila IEEE754

Surse erori

- ***Eroarea relativă*** este:

$\varepsilon_x = |e_x|/|x| = |x - fl(x)|/|x| \le \beta^{e-t}/(0.a_1...a_t...\beta^e)$
$\varepsilon_x \le \beta^{e-t}/(0.10...0\beta^e) = \beta^{1-t}$
$\varepsilon_x \le \beta^{1-t}$ la trunchiere
$\varepsilon_x \le 1/2.\ \beta^{1-t}$ la rotunjire
În general:

$|x - fl(x)|/|x| \le \mu$

de unde deducem:

$fl(x) = x(1+\varepsilon),\ |\varepsilon| \le \mu = K\ \beta^{-t}$

- De exemplu $F(10,4,-20,20)$, $\Omega = 10^{20}(1-10^{-4}) = 9.999 \times 10^{19}$,
- $\omega = 10^{-20-1} = 1.0 \times 10^{-21}$, $\mu_z = 1/2.10^{-4+1} = 5 \times 10^{-4}$

Virgula mobila IEEE754

Propagare erori

Numere aproximative - operaţii exacte

Rezultatul exact al adunării a două numere **x** şi **y**, dacă *operaţiile se execută exact* este **x+y**.

În realitate, se lucrează cu *valorile inexacte* $\underline{x}$ şi $\underline{y}$, în care:

$e_x=|\underline{x}-x|$ şi $e_y=|\underline{y}-y|$

$\underline{x}+\underline{y}=x+y\pm e_{x+y}=x\pm e_x+y\pm e_y=x+y\pm(e_x+e_y)$

$e_{x+y}=e_x+e_y$

$\varepsilon_{x+y}=e_{x+y}/|x+y|=(e_x+e_y)/|x+y|=(|x|\varepsilon_x+|y|\varepsilon_y)/|x+y|$

$\varepsilon_{x+y}=|x|/|x+y|\varepsilon_x+|y|/|x+y|\varepsilon_y=k_x\varepsilon_x+k_y\varepsilon_y$

Virgula mobila IEEE754

Propagare erori

Numere aproximative - operaţii exacte

Pentru scădere:

$\underline{x}-\underline{y}=x-y\pm e_{x-y}=x\pm e_x-(y\pm e_y)=x-y\pm(e_x+e_y)$

de unde:

$e_{x-y}=e_x+e_y$

$\varepsilon_{x-y}=|x|/|x-y|\varepsilon_x+|y|/|x-y|\varepsilon_y=k_x\varepsilon_x+k_y\varepsilon_y$

În acest caz coeficienţii de ponderare:

$k_x=|x|/|x-y|$ şi $k_y=|y|/|x-y|$

pot lua valori foarte mari dacă **x**≈**y**, deci *în cazul scăderii numerelor apropiate* ca ordin de mărime *se pot comite erori foarte mari*

În cazul înmulţirii:

$\underline{x}\underline{y}=xy\pm e_{xy}=(x\pm e_x)(y\pm e_y)=xy\pm xe_y\pm ye_x+e_xe_y=xy\pm(xe_y+ye_x)$

$e_{xy}=xe_y+ye_x$

$\varepsilon_{xy}=\varepsilon_x+\varepsilon_y$

Virgula mobila IEEE754

Propagare erori

Operaţii aproximative - date exacte

Dacă *operaţiile* se reprezintă *aproximativ*, iar *numerele sunt reprezentate exact*, adunarea a două numere $x=f_x.b^{ex}$ şi $y=f_y.b^{ey}$ presupune aducerea celui mai mic (fie acesta **y**) la exponentul celui mai mare, producându-se o *denormalizare*

$fl(x+y)=fl(f_xb^{ex}+f_yb^{-(ex-ey)}b^{ex})=fl[(f_x+f_yb^{-(ex-ey)})b^{ex}]$

$fl(x+y)=fl[(f_x+f_y(1+\mu))b^{ex}]=fl[x+(1+\mu)y]$

Rezultatul operaţiei este *normalizat*:

$fl(x+y)=[x+(1+\mu)y](1+\mu)$

Denormalizarea unuia dintre termeni poate fi evitată dacă se păstrează rezultatul intermediar într-un acumulator cu lungimea **2t** (*acumulator dublu*) În acest caz numai rezultatul final va fi afectat de trunchiere la **t** cifre semnificative şi normalizare, deci:

$fl_2(x+y)=(x+y)(1+\mu)$

Virgula mobila IEEE754

Reprezentarea numerelor reale - Standardul IEEE754

Permite ***reprezentarea realilor*** în:
1) *precizie simplă* `F(2, 24, -126, 127)`, folosind 32 biţi
2) *precizie dublă* `F(2, 53, -1022, 1023)`; se folosesc 64 biţi:
3) *precizie extinsă* `F(2, 65, -16382, 16382)`; se folosesc 80 biţi:
Întrucât $a_1=1$, acesta nu se mai reprezintă (este ascuns), câştigându-se astfel precizie suplimentară. Bitul ascuns este evidenţiat în reprezentarea:

`fl(x)=(-1)^S*2^e.(1.+.f)`

Virgula mobila IEEE754

Reprezentarea numerelor reale - Standardul IEEE754 Precizie simpla

- reprezentare pe 32 biţi
- baza β= 2
- precizie t= 24 biţi (numerele normalizate păstrează numai 23 biţi)
- Numărul real este păstrat prin 3 componente:
  - semnul: 1 bit
  - exponentul: 8 biţi
  - mantissa: 23 biţi (logic24)
- Cei 8 biţi permit: $2^8 = 256$ valori diferite. Domeniul `[0, 255]` este transformat în `[-127, 128]`
- La exponentul (pozitiv sau negativ) se adaugă o valoare constantă care duce la un *exponent deplasat* sau *caracteristică* pozitivă. Factorul de deplasare pentru precizie simplă este127.

Virgula mobila IEEE754

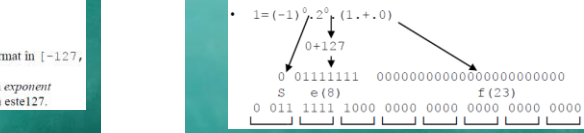
Reprezentarea numerelor reale - Standardul IEEE754 Precizie simpla

- Domeniul deplasat [0-255] reprezintă exponenţi în domeniul [-127, 128]

```
exponent_deplasat = exponent + 127
```

Valoarea numărului este: `V=(-1)^s.2^e.(1.+.f)`

- $1=(-1)^0.2^0.(1.+.0)$



Conditionarea problemelor

- Condiționarea unei probleme caracterizează ***sensibilitatea*** soluției în raport cu erorile din datele de intrare.
- O problemă este ***bine condiționată*** dacă erori mici în date produc de asemeni erori mici în rezultate.
- Condiționarea este o ***proprietate a problemei, independentă de soluția*** aleasă.
- O problemă rău condiționată este „aproape nerezolvabilă" în practică (chiar dacă problema este rezolvată exact, soluția poate fi lipsită de semnificație).
- De exemplu, la evaluarea funcției $y=f(x)$, o perturbare a datelor $x+\Delta x$ produce o perturbare a soluției $y+\Delta y = f(x+\Delta x)$, în care:
- eroarea absolută $|\Delta y| \approx |f'(x)| \cdot |\Delta x|$
- eroarea relativă $\frac{|\Delta y|}{|y|} \approx \frac{|f'(x)|}{|f(x)|} \cdot |x| \cdot \frac{|\Delta x|}{|x|}$
- Problema este *rău condiționată* dacă factorul Lipschitz $L = \frac{|f'(x)|}{|f(x)|} \cdot |x|$ este mare.

Stabilitatea numerica a algoritmilor

- Stabilitatea numerică caracterizează erorile numerice introduse de algoritm, în ipoteza unor date de intrare exacte. Se referă la precizia algoritmului.
- Un algoritm este *instabil* dacă erorile de rotunjire produc erori mari în rezultate.
- Un algoritm numeric stabil nu introduce o sensibilitate suplimentară la perturbații.
- Un algoritm stabil dă rezultate apropiate de soluția exactă pentru o problemă bine condiționată.
- Un algoritm stabil nu poate rezolva o problemă rău condiționată, dar un algoritm instabil poate da soluții slabe chiar pentru o problemă bine condiționată.

Stabilitatea numerica a algoritmilor

Dacă $f: X \to Y$ este o *problemă* și $f^-: X \to Y$ este un *algoritm*, atunci acesta este ***numeric stabil*** dacă pentru $\forall x \in X$, $\exists x^- \in X$, astfel încât:

$$\|f^-(x) - f(x^-)\| = O(\varepsilon_m) \text{ și } \|x - x^-\| = O(\varepsilon_m)$$

Algoritmul f^- destinat rezolvării problemei f este numeric stabil, dacă este îndeplinită una din condițiile:

1. $f^-(x) \equiv f(x)$, adică soluția calculată aproximează bine soluția exactă
2. există x^- apropiat de x astfel încât $f^-(x) = f(x^-)$ – soluția calculată de algoritm cu date de intrare exacte este soluția exactă cu date ușor perturbate.

Exemple de algoritmi instabili:

- inversarea de matrice folosind determinanți
- rezolvarea sistemelor liniare prin factorizare LU fără pivotare
- utilizarea factorizării Cholesky în metoda celor mai mici pătrate (rezultate mult mai bune furnizează factorizarea QR).
- calculul valorilor proprii ca rădăcini ale polinomului caracteristic

Bibliografie

- J. Douglas Faires, Richard L. Burden, Numerical Methods, 4° ed., 2012;
- Richard L. Burden, J. Douglas Faires, Numerical Analysis, 9° ed., 2010;

Curs01:

- V.Iorga, B.Jora, Metode Numerice, Ed. Albastră, 2004;
- Steven G. Krantz, The Proof is in the Pudding, Springer, 2007;
- Chapra, Steven C., Applied numerical methods with MATLAB for engineers and scientists, 3rd ed., McGraw-Hill, 2012;